

Adaptive Partner Modeling for Human–AI Collaboration in Overcooked-Style Multi-Agent Reinforcement Learning

Anonymous submission

Abstract

This temporary draft outlines a research direction for cooperative multi-agent reinforcement learning in Overcooked-style environments. The central hypothesis is that strong human–AI collaboration requires agents that infer a partner’s current goal, adapt to diverse partner policies, and expose interpretable coordination decisions, rather than relying only on self-play against copies of themselves. We propose to study partner-aware policies that combine historical context encoding, goal recognition, optional mixture-of-experts routing, and high-level task abstractions such as symbolic subtasks or planner-backed options. The planned experiments compare self-play, population-trained, communication-enabled, and partner-conditioned agents across layouts, observation modalities, partner types, and adaptation horizons.

Introduction

Cooperative embodied tasks such as Overcooked require agents to coordinate under partial observability, long-horizon credit assignment, and changing partner behavior. Self-play can produce high-performing AI–AI teams, but these policies often assume a narrow distribution of teammates and may fail when paired with humans or unfamiliar agents. The motivating goal of this project is to narrow the gap between AI–AI coordination and human–AI coordination by training agents that can recognize what their partner is trying to do and adapt their own behavior accordingly.

This paper will study Overcooked-style coordination as a testbed for human-compatible multi-agent reinforcement learning. The initial research direction is intentionally broad: we will evaluate whether explicit partner modeling, goal recognition, communication, visual observations, and high-level action abstractions improve zero-shot and few-shot collaboration with diverse teammates.

Draft thesis. A cooperative agent should not only optimize its own policy in self-play; it should maintain a compact model of the partner’s recent behavior, infer likely goals or roles, and condition its actions on that inference. In Overcooked, this means recognizing patterns such as collecting

onions, chopping ingredients, cooking soup, serving dishes, or waiting for handoff opportunities, then selecting complementary behavior.

Planned contributions.

- A partner-aware MARL framework for Overcooked-style environments in JaxMARL.
- A comparison of primitive-action policies and high-level option policies backed by symbolic subtasks or A* planning.
- An empirical study of generalization across layouts, partner policies, observation modalities, and communication settings.
- Diagnostics for partner-goal recognition, adaptation speed, routing behavior, and collaboration quality.

Research Questions

RQ1: Partner adaptation. Can a policy conditioned on recent partner behavior outperform standard self-play when paired with unseen partner policies or humans?

RQ2: Goal recognition. Can explicit prediction of the partner’s current subtask improve coordination, interpretability, or sample efficiency?

RQ3: Policy specialization. Does a mixture-of-experts or personality-conditioned policy improve robustness to different teammate styles compared with a single monolithic policy?

RQ4: Observation modality. Do image-based observations improve transfer and generalization compared with structured feature-vector observations, or do they mainly increase optimization difficulty?

RQ5: Action abstraction. Does replacing low-level movement actions with high-level subtasks such as “go to onion,” “chop onion,” or “serve soup” make coordination easier to learn?

RQ6: Communication. When and how should agents communicate: every timestep, only under uncertainty, through learned continuous messages, or through human-interpretable goal messages?

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Background and Related Work

Overcooked and cooperative MARL. Overcooked-style environments are useful because they require division of labor, sequencing, handoffs, collision avoidance, and adaptation to a partner’s choices. This draft will later position the project relative to Overcooked-AI, JaxMARL, cooperative gridworlds, and recent collaborative Overcooked benchmarks.

Self-play, population training, and zero-shot coordination. Self-play creates strong conventions but may overfit to the policy distribution seen during training. Population-based training and diverse partner pools can expose agents to a wider range of teammate behavior, but they may still lack explicit mechanisms for recognizing partner intent.

Goal recognition and theory of mind. The notes suggest a goal-recognition framing: observe a sequence of partner actions and classify the partner’s likely subtask. This connects to symbolic planning, Bayesian delegation, context grammars, and theory-of-mind-inspired partner modeling.

Personality and mixture-of-experts policies. Prior work such as personality-conditioned cooperation and mixture-of-experts routing motivates learning separate behavioral experts for different partner styles. In this project, “personality” can initially be treated operationally as a latent cluster of partner policies or subtask preferences rather than a psychological claim.

Communication in MARL. Learned communication methods such as centralized message generators, gated messages, and intention sharing motivate experiments where agents exchange compact coordination signals. A key open question is whether communication should be continuous and learned, discrete and interpretable, or derived from goal-recognition labels.

Vision-language-action and symbolic abstractions. The longer-term direction includes visual inputs and language-like task commands. For the temporary paper, this is scoped as an action abstraction question: compare low-level actions against planner-backed options or labeled subtasks before attempting full vision-language-action training.

Problem Setting

We consider a two-agent cooperative cooking environment. Agents receive local or global observations and jointly maximize team reward by completing recipes. A trajectory consists of states, observations, actions, rewards, and optional messages. The partner may be a self-play teammate, a policy sampled from a partner population, a scripted or planner-based agent, a subtask-specialized policy, or eventually a human.

Environment. The initial implementation target is JaxMARL Overcooked. Planned layout families include simple coordination layouts, forced-division-of-labor layouts, long-horizon layouts, and layouts with bottlenecks or handoff points.

Partner types. Candidate partner distributions include:

- self-play partner policies;
- population-trained PPO or MAPPO policies;
- subtask-biased policies such as onion collector, chopper, cooker, server, or dish manager;
- scripted planner partners with known goals;
- noisy or delayed partners for robustness testing;
- human partners in a later evaluation stage.

Observation types. We will compare structured feature vectors, grid/tensor observations, and image-like observations. The first experiments should keep the structured observations as a stable baseline, then add visual observations only after the partner-modeling pipeline works.

Proposed Method

Partner-Aware Policy

The main policy conditions on both the current observation and a compact embedding of recent partner behavior. A context encoder consumes the last K timesteps of partner observations, actions, positions, interactions, and optional messages. The resulting embedding is used by the actor, critic, goal-recognition head, and optional expert router.

Inputs.

- current agent observation;
- recent partner action and interaction history;
- recent environment events, such as pickups, chops, cooking starts, deliveries, and failed interactions;
- optional message history;
- optional layout identifier or layout embedding.

Outputs.

- primitive action or high-level option;
- predicted partner subtask;
- optional communication message;
- optional expert-routing distribution.

Goal Recognition Head

The goal-recognition module predicts the partner’s current or next subtask from recent context. Initial labels can come from scripted policies, planner rollouts, or subtask-specialized PPO policies. Later, pseudo-labels can be inferred from state transitions, such as moving toward onion stations, cutting boards, pots, dishes, or serving counters.

Candidate goal labels.

- get onion;
- bring onion to pot or cutting board;
- chop ingredient;
- start or monitor cooking;
- get dish;
- serve soup;
- clear path or reposition;
- wait for partner handoff.

Expert Routing

The mixture-of-experts version maintains a set of policy experts and a router conditioned on partner context. Experts may specialize by partner style, layout family, role, or subtask. A simple first version can route over independently trained policies; a later version can train the router and experts end-to-end with load-balancing regularization.

High-Level Options

Instead of directly predicting primitive actions at every timestep, the policy may choose from planner-backed options. For example, selecting “go to chopping board” invokes a shortest-path or A* executor until the option terminates or becomes invalid. This separates strategic coordination from low-level navigation and may improve interpretability.

Communication Variants

Communication will be evaluated as an intervention rather than assumed necessary. Candidate variants are:

- no communication;
- learned continuous messages;
- discrete goal messages predicted by the goal-recognition head;
- uncertainty-triggered messages;
- human-readable messages for later human studies.

Training Plan

Stage 1: Baselines. Train self-play agents on a fixed set of layouts. Measure AI–AI performance, zero-shot cross-play with independently trained agents, and generalization to held-out layouts.

Stage 2: Partner pool. Train or script a diverse partner pool, including subtask-biased agents and policies with different conventions. Use this pool to evaluate whether the main agent can adapt to unseen teammate behavior.

Stage 3: Goal recognition. Train a context encoder and goal-recognition head using labeled or pseudo-labeled partner trajectories. Evaluate prediction accuracy and whether auxiliary goal loss improves control performance.

Stage 4: Partner-conditioned control. Train actor-critic policies conditioned on the partner embedding. Compare a single conditioned policy, a mixture-of-experts policy, and an oracle-conditioned upper bound.

Stage 5: Options and communication. Add high-level option policies and communication variants. Run ablations to determine whether performance gains come from partner modeling, action abstraction, message passing, or additional supervision.

Experiments

Experiment 1: Self-Play Generalization

Question. How well do self-play agents generalize across random seeds, partner policies, and layouts?

Design. Train independent self-play agents and evaluate cross-play pairings. Include held-out layouts and partner policies trained with different seeds or reward shaping.

Expected outcome. This establishes the baseline gap between AI–AI self-play performance and coordination with unfamiliar partners.

Experiment 2: Partner-Goal Recognition

Question. Can recent behavior predict the partner’s current or next subtask?

Design. Collect trajectories from scripted, planner-based, and subtask-biased policies. Train context encoders with different history lengths and compare prediction accuracy, calibration, and confusion between similar tasks.

Ablations.

- context length K ;
- action-only history versus action plus state-event history;
- feature-vector observation versus visual observation;
- supervised labels versus pseudo-labels.

Experiment 3: Partner-Conditioned Policy

Question. Does conditioning on partner context improve team reward with unseen partners?

Design. Compare self-play PPO, population-trained PPO, context-conditioned PPO, and context-conditioned actor-critic variants. Evaluate on held-out partner policies, layouts, and noisy partner behavior.

Metrics. Report team reward, soups delivered, collision rate, idle time, redundant task attempts, recovery from partner mistakes, and adaptation time after partner behavior changes.

Experiment 4: Mixture-of-Experts Routing

Question. Does explicit expert routing improve adaptation beyond a single conditioned network?

Design. Train or initialize experts for different partner styles or subtasks. Compare dense policy conditioning, hard expert routing, soft expert routing, and oracle expert labels.

Diagnostics. Measure expert utilization, routing entropy, route stability, and whether experts align with interpretable partner styles.

Experiment 5: Action Abstraction

Question. Do high-level options make coordination easier than primitive actions?

Design. Compare primitive-action control with policies that select symbolic subtasks executed by A* or another planner. Measure sample efficiency, final reward, failure modes, and robustness when the planner path is blocked by the partner.

Experiment 6: Communication

Question. When does communication help, and what form should it take?

Design. Compare no communication, learned continuous communication, discrete goal messages, and uncertainty-triggered communication. Evaluate both performance and interpretability.

Experiment 7: Human or Human-Proxy Evaluation

Question. Do partner-aware agents improve human–AI collaboration?

Design. If human evaluation is feasible, pair trained agents with human players. If not, use human-proxy policies with behavior distributions designed to mimic common human strategies and mistakes.

Metrics. Measure reward, task completion, interruptions, partner burden, predictability, and subjective preference for human trials.

Evaluation Metrics

- **Task performance:** team reward, soups delivered, completion time, and success rate.
- **Coordination quality:** idle time, redundant actions, collisions, failed handoffs, and blocked paths.
- **Generalization:** performance on held-out layouts, seeds, partner policies, and recipe variants.
- **Adaptation:** reward in the first N timesteps with a new partner and recovery after partner-policy switches.
- **Interpretability:** goal-recognition accuracy, expert-route consistency, and agreement between messages and behavior.
- **Efficiency:** environment steps, wall-clock training time, and sensitivity to hyperparameters.

Anticipated Results and Claims

This section will be rewritten after experiments. The current hypotheses are:

- self-play will perform well with matched partners but degrade under cross-play and human-proxy evaluation;
- context-conditioned partner modeling will improve zero-shot coordination with diverse partners;
- goal-recognition supervision will improve interpretability and may improve sample efficiency;
- mixture-of-experts routing will help if partner styles are meaningfully separable, but may collapse without balancing or sufficient diversity;
- high-level options will simplify long-horizon coordination but may fail when the planner cannot account for partner interference;
- communication will help most in partial-observation or role-asymmetric layouts, and may be unnecessary in simple layouts.

Limitations and Risks

- The partner-style labels may be too artificial if generated only from scripted or subtask-biased policies.
- Vision observations may add training cost without testing the core partner-modeling hypothesis.
- Mixture-of-experts policies may underperform dense policies if routing creates brittle specialization.
- Planner-backed options may hide important coordination failures by solving low-level navigation externally.
- Human studies may be expensive, noisy, and require interface work outside the core MARL pipeline.
- Claims about human compatibility should be conservative until tested with real human partners.

Implementation Plan

Immediate milestone. Produce a stable self-play baseline in JaxMARL Overcooked, save evaluation rollouts, and define the event extraction code needed for partner-goal labels.

Next milestone. Build partner-policy diversity: independent seeds, shaped rewards, scripted subtask agents, and simple planner-based policies.

Modeling milestone. Train the context encoder and goal-recognition head, then plug the embedding into a policy. Keep the first version simple before adding mixture-of-experts routing.

Experiment milestone. Run the experiment matrix across layouts and partners. Prioritize the comparisons that directly answer whether partner modeling improves zero-shot coordination.

Writing milestone. Replace each TODO and hypothesis with measured results, plots, qualitative rollout analysis, and citations to the relevant prior work.

Paper TODOs

- Decide whether the first submission target is a workshop-style project paper or a full conference paper.
- Add precise citations for Overcooked-AI, JaxMARL, collaborative Overcooked, goal recognition, MoP-SAN/personality modeling, HAMMER, IC3Net, intention sharing, VLA methods, and CARBS.
- Define exact environment layouts and partner-policy generation procedure.
- Add architecture diagram for partner-aware policy and goal-recognition head.
- Add experiment table with final baselines, metrics, and compute budget.
- Add result figures once baseline runs are complete.
- Update title after the core contribution is clearer.

Conclusion

This temporary paper frames the project around adaptive partner modeling for Overcooked-style MARL. The working claim is that human-compatible cooperation requires explicit reasoning about teammate behavior: recognizing goals, adapting to partner style, choosing complementary subtasks, and optionally communicating intent. The next step is to establish self-play and partner-diversity baselines, then test whether context-conditioned policies and high-level abstractions close the coordination gap with unfamiliar teammates.